Programación Orientada a Objetos en C++: Desarrollo de un Simulador de Combate Espacial en Consola (class Nave)

Autores: Del Pino, Mattia, Orue, Ferrari

E.E.S.T. N.º 1 “Eduardo Ader”, Vicente López

Laboratorio de Programación (LPR), 5.º año 3.ª, Grupos A-B

Profesor: Ganduglia Yamil y Mansilla Muñoz York Elías

Actividad 8, 2.º cuatrimestre 2026

21 de octubre de 2026

Introducción

El presente informe técnico documenta el desarrollo de la Actividad 8 de la materia Laboratorio de Programación, cuyo objetivo es aprender los fundamentos de la Programación Orientada a Objetos (POO) en C++ mediante la construcción de un simulador de combate espacial en consola. El proyecto parte de un motor de juego base (archivo `src/main.cpp`) que define tres clases: `Nave` (el jugador), `Asteroide` (el obstáculo) y `Bala` (el proyectil). El jugador controla la nave con las teclas W, A, S y D, dispara con la barra espaciadora y pierde una vida cada vez que un asteroide lo impacta.

La actividad representa la evolución natural del concepto de `struct` trabajado en la Actividad 7. Mientras que en la programación estructurada los datos se agrupan pero las funciones que los modifican quedan sueltas en el código, en la POO los datos (atributos) y las funciones (métodos) se reúnen en una misma unidad llamada objeto, y se protegen mediante el encapsulamiento.

Se trabajó con la metodología 5R (Registrar, Reducir, Responder, Reflexionar y Revisar): se transcribió y compiló el código base, se identificaron las secciones `private:` y `public:`, se respondió el cuestionario técnico, se implementó una funcionalidad adicional y se publicó el proyecto en los repositorios de GitHub y GitLab con la estructura de carpetas exigida (`juegodenaves/` con `docs/`, `src/` y `capturas/`).

El informe se organiza en Desarrollo (respuestas al cuestionario técnico), Conclusión y Referencias.

Desarrollo

1. struct vs class

En C++ un `struct` y una `class` son técnicamente casi idénticos: ambos pueden contener atributos y métodos, constructores y modificadores de acceso. La diferencia técnica principal es el nivel de acceso por defecto: en un `struct` todos los miembros son `public` salvo que se indique lo contrario, mientras que en una `class` todos los miembros son `private` por defecto.

En la Actividad 7 el `struct` se utilizó para agrupar variables (por ejemplo, id, nombre y resultado), todas accesibles desde cualquier parte del programa. En la clase `Nave`, en cambio, los atributos `x`, `y` y `vidas` se declaran en la sección `private:`, por lo que solo pueden ser leídos o modificados a través de los métodos de la sección `public:` (`getX()`, `getY()`, `getVidas()`, `reducirVida()` y `mover()`). Además, la clase incorpora comportamiento propio: la nave sabe pintarse, borrarse y moverse por sí misma (`jugador.mover(tecla)`), algo que en el enfoque estructurado requeriría funciones externas sueltas.

2. Encapsulamiento

El encapsulamiento consiste en ocultar los datos internos de un objeto y exponer únicamente métodos públicos que actúan como “puertas controladas”. Dejar una variable como la salud (en este proyecto, `vidas`) como pública es peligroso porque cualquier parte del programa podría modificarla libremente, sin ninguna validación.

Un programador novato podría escribir desde `main()` una instrucción como `jugador.vidas = -50;` o `jugador.vidas = 9999;`, dejando a la nave con salud negativa o con vidas infinitas, lo que rompería la lógica del juego. Por ejemplo, la condición de fin de partida `if (jugador.getVidas() == 0)` nunca se cumpliría si las vidas pasaran a ser negativas, y el juego no terminaría. Al mantener `vidas` como `private`, el único modo de alterarla es mediante `reducirVida()`, y la clase puede incluir una validación para impedir valores negativos. Esto centraliza las reglas en un solo lugar, facilita encontrar errores y evita efectos colaterales inesperados.

En el código base, `x` e `y` también son privados: la posición de la nave solo cambia mediante `mover()`, que además impone los límites de pantalla (`x > 2`, `x < 70`, `y > 3`, `y < 20`).

3. Constructor

Un constructor es un método especial que tiene el mismo nombre que la clase, no declara tipo de retorno y se ejecuta automáticamente al crear un objeto. Su función es inicializar los atributos para que el objeto nazca en un estado válido y conocido.

En el código provisto, el constructor de la clase es `Nave(int _x, int _y, int _vidas)`, que asigna `x = _x; y = _y; vidas = _vidas;`. Al ejecutar `Nave jugador(35, 20, 3);` en `main()`, se crea la nave en la posición (35, 20) y con 3 vidas. Sin constructor, los atributos tendrían valores basura (memoria sin inicializar). Una forma similar, `Nave(string _nombre, ...)`, mencionada en el enunciado, funciona bajo el mismo principio: recibe parámetros (el prefijo guion bajo distingue el parámetro del atributo) y los copia a los atributos privados. Las clases `Asteroide` y `Bala` utilizan constructores equivalentes para recibir su posición inicial.

4. Desafío de funcionalidad

De las 15 funcionalidades del anexo se eligió el menu de inicio.

Descripción: Muestra una pantalla de presentación estilizada en la consola antes de que comience el bucle principal del juego. Utiliza posicionamiento de cursor **(`gotoxy`) para** centrar el título del simulador, solicita al usuario el ingreso de su nombre mediante **`getline`** y detiene la ejecución del programa con **`cin.get()`** a la espera de que el piloto presione la tecla **`ENTER`** para despegar y dar inicio a la simulación.

Partes del código modificadas. Se insertó **al comienzo de la función principal `int main()`**, de manera estrictamente previa a la inicialización del bucle principal del juego while (!gameOver) y antes de la instanciación de los objetos de la clase Nave y los vectores de la clase Asteroide.

Detalle de la modificación: Se reemplazó el bloque de lectura simple de datos inicial por un menú de bienvenida interactivo. Este bloque hace uso de system("cls") para limpiar la pantalla, de la función de posicionamiento gotoxy(25, y) para centrar las etiquetas de texto de la interfaz, de getline(cin, nombrePiloto) para capturar el nombre del operador, y de cin.get() para detener la ejecución y aguardar a que el usuario presione la tecla **ENTER** para despegar y arrancar la simulación.

Verificación. Procedimiento de prueba: Se compiló el código fuente a través del entorno de MinGW (g++) en la terminal de PowerShell y se ejecutó el binario resultante en la consola.

Resultado observado: Se comprobó visualmente que la aplicación inicia desplegando el título centrado y la solicitud de credenciales de piloto. Tras ingresar el nombre y presionar ENTER, la consola limpia la interfaz y da paso de forma fluida al bucle de combate, renderizando correctamente el HUD superior con las vidas y el nombre del piloto en pantalla.

Evidencia gráfica: La prueba de ejecución exitosa y la correcta visualización del HUD con los datos del usuario se encuentran documentadas en la captura de pantalla alojada en el repositorio.

Conclusión

La actividad permitió comprender que la POO no es un concepto ajeno a lo visto anteriormente, sino la evolución del `struct`: los datos y las funciones que los manipulan conviven en una misma clase, y el control de acceso (`private` y `public`) protege la integridad del estado del objeto. El motor del juego mostró de forma práctica los tres pilares trabajados: clases como moldes de objetos (`Nave`, `Asteroide`, `Bala`), constructores para una inicialización segura y encapsulamiento mediante métodos de acceso.

Como observación técnica, el código base utiliza memoria dinámica (`new`) para asteroides y balas. Al reemplazar un asteroide (`asteroides[i] = new Asteroide(...)`) sin liberar el anterior con `delete`, se genera una pérdida de memoria; además, borrar elementos de un `vector` dentro de un bucle por índice requiere cuidado para no omitir elementos. Estos puntos son oportunidades de mejora para futuras versiones del proyecto.

Finalmente, la documentación (informe APA v7, manuales del programador y del usuario, README y CHANGELOG) y el uso de repositorios en GitHub y GitLab refuerzan una práctica profesional: si el código funciona, debe estar documentado y versionado.

Referencias

GeeksforGeeks. (s. f.). *Access modifiers in C++*. https://www.geeksforgeeks.org/access-modifiers-in-c/

Microsoft. (s. f.). *Constructores (C++)*. Microsoft Learn. https://learn.microsoft.com/es-es/cpp/cpp/constructors-cpp

The C++ Resources Network. (s. f.). *Classes (I)*. cplusplus.com. https://cplusplus.com/doc/tutorial/classes/

[Ferrari.U]. (2026). *juegodenaves*

GitHub.https://github.com/Ferraribip/juegodenaves.git